AMRUTVAHINI COLLEGE OF ENGINEERING, SANGAMNER

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

2026-2027

Project Synopsis
on

"Design and Implementation of an LLM-Orchestrated Multi-Agent System for Constraint-Aware Personalized Trip Itinerary Planning"

BE AI & DS

BY

Group Id-

Mr. Satyam Kadlag (Roll No. 1517)

Mr. Mayuresh Kakade (Roll No. 1518)

Mr. Yash Kamble (Roll No. 1519)

Mr. Navnath Mate (Roll No. 1528)

Prof. S. N.
Project Guide
Dept. of AI&DS

Prof. Bramhane V. V.
Project Coordinator
Dept. of AI & DS

Dr. Panhalkar A. R.
H.O.D
Dept. of AI & DS

AMRUTVAHINI COLLEGE OF ENGINEERING, SANGAMNER

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

417527: Project Stage I

PROJECT TITLE

Design and Implementation of an LLM-Orchestrated Multi-Agent System for Constraint-Aware Personalized Trip Itinerary Planning

SYNOPSIS REPORT

(Project Work Stage – I)

Chapter 1: Introduction

1.1 Introduction

Personalized trip planning is a multi-constraint engineering problem rather than a simple search task. A traveler may state a destination, origin, dates, budget, group composition, preferred activities, food, accommodation, transport, accessibility needs, and daily time limits in natural language. A useful plan must then reconcile these preferences with point of interest (POI) opening hours, visit duration, travel time, distance, cost, and changing context while arranging the selected places into a coherent day-wise itinerary.

A Large Language Model (LLM) can interpret a loosely expressed request and produce explanations, but a reliable planner also needs external data, deterministic checks, and optimization. The proposed system therefore uses a Multi-Agent System (MAS) in which specialized logical agents handle profile extraction, evidence retrieval, routing, budgeting, scheduling, context, synthesis, and verification. Retrieval-Augmented Generation (RAG) will ground recommendations in retrieved POI and travel records. This decomposition follows the strengths demonstrated by tool-assisted trip planning [1], distributed co-modal planning [2], and multi-objective itinerary optimization [4], while reserving final control for an orchestrator rather than allowing unrelated agent outputs.

1.2 Problem Statement

Conventional travel applications commonly solve isolated subtasks such as destination search, hotel discovery, route lookup, or generic recommendation. They may not jointly interpret nuanced natural-language preferences, allocate activities over several days, honor budgets and opening windows, account for accessibility, and minimize unproductive travel between POIs. Information is often fragmented across maps, review sites, transport services, weather sources, and booking platforms, leaving the traveler to reconcile inconsistencies manually.

An unverified LLM-only solution adds a further risk: it can recommend unsupported or closed venues, underestimate transfer time, overlook a daily limit, duplicate activities, or create a schedule that cannot be executed. The engineering problem is therefore to design an orchestrated, tool-assisted multi-agent framework that converts a request into structured constraints, retrieves

evidence, generates and optimizes a feasible itinerary, validates each operational condition, and triggers targeted replanning when a conflict is found.

1.3 Aim

To design and implement an LLM-orchestrated multi-agent system that coordinates specialized planning, retrieval, routing, budgeting, scheduling, and verification agents to produce personalized, constraint-aware, and feasible day-wise trip itineraries.

1.4 Objectives

1. Capture travel requirements from natural-language input and convert them into a validated, structured user profile.
2. Design a modular multi-agent architecture with explicit responsibilities, communication contracts, shared state, and replanning flow.
3. Retrieve and rank contextually relevant POIs, accommodation, food, and transportation options from configured data sources.
4. Generate a day-wise itinerary that balances preference satisfaction, budget, time, and travel distance under operational constraints.
5. Validate factual consistency, schedule feasibility, route efficiency, and constraint satisfaction before presenting the itinerary.

1.5 Motivation

Travel planning is motivated by a practical information-overload problem. A user may compare hundreds of places, reviews, routes, fares, schedules, and weather conditions, yet a locally attractive choice can make the complete plan infeasible. Manual planning is time-consuming because each change to one day may affect the remaining budget, travel sequence, hotel location, and availability of later activities.

Collaborative AI agents provide an opportunity to separate these concerns while preserving a unified plan. The LLM-TripPlanner study shows the value of user profiling, tools, urban data, and RAG [1]; the co-modal ATIS study demonstrates distributed route-agent coordination across time, cost, and emissions [2]; and the ACO study shows how preference satisfaction, cost, visit order, and historical trips can be treated jointly [4]. These findings motivate a

modular prototype that combines semantic interaction with explicit feasibility checks and explainable revision.

1.6 Scope of the Project

The Stage-I scope covers the design and prototype implementation of a planning assistant for representative personal, family, and small-group travel queries. It will emphasize traceable data flow and constraint handling rather than commercial booking operations.

Included scope:

- Natural-language capture of destination, origin, dates, duration, budget, travelers, interests, food, accommodation, transport, and mobility requirements.
- Structured profile extraction with missing-field detection, normalization of dates and currency, and separation of hard constraints from soft preferences.
- Retrieval, filtering, and semantic ranking of POIs, restaurants, accommodation candidates, and transport options from configured sources.
- Day-wise itinerary generation with estimated arrival, visit, meal, transfer, and rest periods.
- Route sequencing using map-derived distance and travel-time estimates, with a heuristic optimizer for visit order.
- Checks for budget, opening hours, daily time limits, accessibility, travel mode, excessive detours, duplicate POIs, and incompatible time windows.
- Evidence-linked explanations for recommendations, assumptions, estimates, and any unresolved data limitations.
- Partial or complete replanning when the user changes a preference or the verification agent reports a conflict.
- Prototype evaluation with representative travel queries, feasibility checks, constraint-violation counts, and comparison with a simpler single-agent or rule-based baseline.

- Persistence of user profiles, candidate evidence, itinerary versions, validation status, and feedback for reproducibility.

Initial exclusions and limitations:

- The prototype will not purchase tickets, reserve rooms, process payments, or guarantee availability.
- Visa processing, insurance underwriting, and legal travel authorization will remain outside scope.
- Real-time traffic, weather, transit, and pricing will be used only when an appropriate API or dataset is configured; otherwise the system will label estimates and assumptions.
- Emergency travel management, medical dispatch, and safety-critical navigation will not be provided.
- Initial coverage may be limited to selected cities and available languages rather than guaranteed global completeness.
- Reinforcement learning will be treated as a literature-informed future option, not as a claimed component of the Stage-I prototype.

Target users: Independent travelers, families, small groups, and academic evaluators who need an editable, explainable itinerary rather than an automated booking service.

Evaluation boundary: Expected outcomes will be assessed through prototype behavior and representative queries; no deployment, commercial API acquisition, or completed user study is claimed in this synopsis.

Chapter 2: Literature Review

2.1 Literature Survey

Sr. No.	Author(s) & Year	Methodology / Techniques Used	Findings	Limitations
1.	Y. Liu et al. (2025) [1]	End-to-end LLM agent with GPT-4o profile and generation modules; Map API validation; RAG over a Suzhou urban feature database built from 1,259,360 street-view images plus traffic data; PSPNet, ConvNeXt, LLaVA, and dynamic route weighting.	A 237-traveler evaluation reported satisfaction above 60% in nearly all dimensions, above 70% for transport-mode alignment, 63.2% overall satisfaction, and 58.1% satisfaction with destinations.	No seamless integration of several travel modes in one trip; occasional LLM hallucinations affected destination recommendations; structured POI data and further reliability improvements were left for future work.
2.	M. Dotoli et al. (2017) [2]	True multi-agent ATIS using interface, super, transport-service, transport-information, route, and optimizer agents; co-modal and transfer graphs; parallel Dijkstra searches; route-agent coalitions; genetic user-vehicle assignment; weighted time, cost, and emissions criteria.	Lille-area urban and extra-urban cases showed that routes changed with criterion priority and service disruption, multiple requests could share services, and the reported PC implementation returned the studied scenarios in under three minutes.	Depends on transport-operator data sharing, reliable and timely updates, and organizational/legal cooperation; message/coalition complexity and computational cost remain concerns; natural-language and semantic personalization are limited.
3.	A. Skrynnik et al. (2021) [3]	Partially observable four-connected-grid multi-agent pathfinding; hybrid policy mixes curriculum-trained PPO for goal reaching with deep MCTS or QMIX for conflict	Hybrid policies outperformed standalone policies on cooperative individual and cooperative success metrics; MCTS plus curriculum PPO was generally the	Simplified grid motion omits general graphs and vehicle kinematics; the policy mixer is simple and non-trainable. The work concerns

Sr. No.	Author(s) & Year	Methodology / Techniques Used	Findings	Limitations
		resolution; evaluated in the POGEMA simulator on maps from 8x8 to 32x32 with 2-16 agents.	strongest combination in difficult cooperative settings.	autonomous-agent collision-free routing, not tourist itinerary generation.
4.	E. Saeki et al. (2022) [4]	Multi-objective ACO treats past trips as pheromone evidence and assigns ants to satisfaction, cost, similarity, and balanced objectives; uses local search, Levenshtein/LCS route similarity, Pareto solutions, path relinking, and pheromone updates on Kyoto, Sapporo, and Tokyo data.	The method outperformed P-VNS and MOACOPW on POI satisfaction, cost, and past-trip similarity, produced more stable/diverse solutions, and received higher route scores in a 20-participant study for each target area.	Limited to one-day planning; hotel selection and multi-day support are absent; time-window constraints such as shortened business hours were identified as future work; effectiveness depends on suitable historical trip records.
5.	J. Yin et al. (2025) [5]	Two-stage inter-region/intra-region vehicle routing; road-network partitioning; asynchronous actor-critic MAPPO with centralized training/decentralized execution, low-dimensional global state, asynchronous trajectory collection, GRU request encoding, and reachability-DAG action masking.	SUMO experiments on a synthetic network and a Cologne network with 2,515 intersections and 5,784 road segments showed better throughput, travel time, and CO2 results than five baselines, with improved traffic load balance.	Targets cooperative vehicle routing rather than human tourism itineraries; uses static topology-based regions, omits explicit queue and traffic-light observations, and requires training/simulation resources and credible traffic data.

2.2 Research Gap

The surveyed work addresses complementary parts of the planning problem, but no single study combines every capability required by the proposed project. LLM-TripPlanner interprets preferences and grounds route recommendations

with urban data, yet the paper reports residual hallucination risk and no seamless combination of multiple travel modes within one trip [1]. Its three-module agent is valuable evidence for tool-grounded personalization, but it does not provide the explicit multi-agent specialization and comprehensive itinerary constraint loop proposed here.

The co-modal ATIS is a genuine MAS: route, transport-service, information, interface, and optimizer agents cooperate through graphs and coalitions to optimize time, cost, and emissions for several users [2]. Its distributed design informs orchestrator and route-agent responsibilities. However, the system predates modern LLM interaction and does not provide semantic preference extraction, RAG, conversational feedback, or tourism-specific POI and day-schedule reasoning.

ACO-based trip planning directly optimizes POI satisfaction, cost, visiting order, and similarity to past trips [4]. It offers an important basis for multi-objective candidate selection and visit-order heuristics. Nevertheless, the evaluated method is limited to one-day trips, does not select hotels, and identifies opening-hour time windows as future work, leaving a gap for multi-day scheduling with operational constraints.

Hybrid Policy Learning addresses cooperative multi-agent pathfinding under partial observability [3]. Its separation of goal-reaching and conflict-resolution behavior suggests that complex planning can benefit from modular policies. The agents are autonomous entities moving on simplified grids, however, so its collision-avoidance results cannot be presented as evidence of personalized tourist-itinerary quality.

Asynchronous MARL coordinates vehicle routes on capacity-constrained road networks and contributes ideas for scalable regional planning, asynchronous decisions, critic-based cooperation, and loop avoidance [5]. It is not a tourism recommender and does not model POI relevance, meals, accommodation, traveler preferences, opening hours, or budget. Its learning and simulation requirements also make it a future routing option rather than a necessary Stage-I component.

Across the five studies, recommendation, route planning, scheduling, budget management, semantic interaction, and verification are largely handled as separate problems. Numerical optimizers can produce feasible sequences but may not understand nuanced language; LLM agents can interpret nuance but

require evidence and constraint checking; vehicle pathfinding coordinates motion but does not construct a human-centered itinerary.

The proposed project intends to address this combined gap through an orchestrated MAS that maintains one structured trip state, uses an LLM for controlled interpretation and synthesis, retrieves evidence through RAG and travel tools, assigns routing and feasibility checks to specialized agents, and requires a critic to validate facts and constraints before release.

Hard constraints will be checked before soft preference optimization, and every conflict will include a reason and an owner for revision. This design prevents agents from returning unrelated prose and allows the supervisor to request only the affected subtask, such as replacing a closed POI or resequencing a day after a route change.

Because this is a Stage-I synopsis, these features are proposed design decisions and expected engineering contributions. Their quality will require prototype implementation and formal evaluation against simpler single-agent and non-agent baselines; experimentally proven superiority is not claimed.

Chapter 3: System Architecture

3.1 Proposed System

The proposed system will accept a natural-language trip request through a web interface and will convert it into a structured planning job. The Supervisor/Orchestrator Agent will validate essential fields, establish the hard and soft constraints, create a common trip state, decompose the job, and invoke specialized agents in a controlled sequence. Agent messages will use typed JSON/Pydantic objects rather than unrestricted conversational text so that evidence, estimates, assumptions, conflicts, and validation decisions can be inspected and reproduced.

The orchestrator will be responsible for state transitions and replanning. It will not simply concatenate responses: it will wait for required evidence, apply dependency rules, reconcile conflicting outputs, and permit the Itinerary Synthesis Agent to assemble a plan only after routing and constraint data are available. The following logical agents are proposed; they represent modules and do not imply that every module has already been implemented.

- **Supervisor/Orchestrator Agent:** Parses the job envelope, assigns subtasks, tracks dependencies and deadlines, records provenance, resolves ownership of conflicts, and initiates partial or full replanning.
- **User Profile and Preference Agent:** Extracts destinations, dates, group details, interests, food, accommodation, transport, accessibility, budget, and daily limits; identifies missing or ambiguous values; and classifies constraints as hard or soft.
- **Destination and POI Retrieval Agent:** Queries configured POI, restaurant, hotel, and activity sources; records source identifiers and retrieval time; filters obvious mismatches; and returns normalized candidates with evidence.
- **Transportation and Route Planning Agent:** Obtains distance and travel-time estimates for permitted modes, builds a route matrix, proposes efficient visit orders, and flags transfers or detours that make a day impractical.
- **Budget and Constraint Agent:** Aggregates estimated costs, checks opening windows, daily duration, accessibility, budget, visit duration, and

mode restrictions, and produces machine-readable pass/fail results with explanations.

- **Context Agent:** Retrieves weather, traffic, or disruption information only when a configured source is available; otherwise it marks the contextual field unavailable rather than guessing.
- **Itinerary Synthesis Agent:** Combines validated candidates and route data into a coherent day-wise schedule with arrival, activity, meal, transfer, rest, estimated cost, and rationale fields.
- **Critic/Verification Agent:** Performs the final independent check for unsupported claims, duplicate POIs, closed venues, time overlap, excessive travel, budget violations, missing evidence, and inconsistent totals.
- **Shared memory and common trip state:** Stores the user profile, candidate POIs, evidence metadata, coordinates, opening hours, costs, travel modes, route matrix, daily allocations, conflicts, itinerary versions, and validation status.
- **External data and tools:** May include a relational database, vector index, maps/places/directions services, transit and weather sources, and an LLM API. Commercial services are proposed options and are not claimed to be acquired.
- **Conflict resolution and replanning:** Hard-constraint failures block publication. The critic returns each conflict to the orchestrator with the affected day/POI and requested repair; the supervisor then re-invokes only the relevant retrieval, route, budget, or synthesis agent.
- **Final output:** A validated day-wise itinerary will show assumptions, estimated travel and cost, supporting evidence where available, warnings for uncertain data, and a concise explanation of how the plan reflects the user profile.

3.2 Technology Stack

Proposed Technology Stack

The following stack is a feasible prototype configuration. Equivalent tools may be substituted after feasibility, licensing, and API-access checks; no paid service or proprietary dataset is assumed to be available at this stage.

Component	Technology
Frontend (UI)	React or Next.js web interface with TypeScript
Backend	FastAPI REST service with asynchronous orchestration workers
Programming Language	Python 3.x for backend/agents; TypeScript for the web client
Database	PostgreSQL with pgvector for trip state, profiles, evidence, and embeddings; optional Redis cache
Framework / Libraries	LangGraph or equivalent, Pydantic, SQLAlchemy, NetworkX, OR-Tools, and sentence-transformers
APIs / Tools (if any)	LLM API, Maps/Places/Directions, optional transit/traffic and weather APIs, Docker, pytest, and Git

3.3 System Architecture Diagram

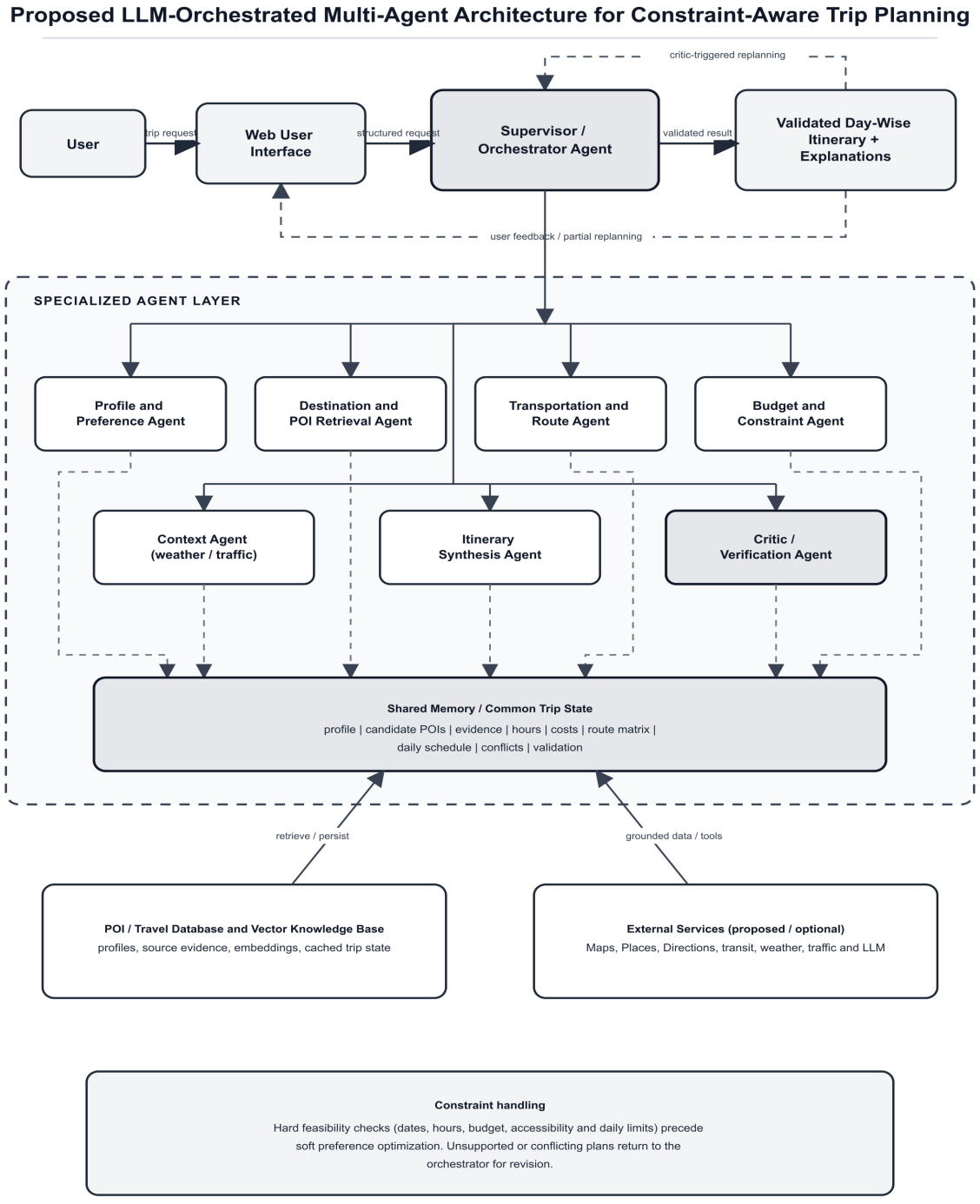


Figure 1. Proposed multi-agent architecture and validation/replanning flow.

Chapter 4: Methodology

4.1 Proposed Methodology

1. The user enters the destination, origin, dates, duration, budget, group details, interests, food, accommodation, transport, accessibility, and other preferences through the web interface.
2. The backend validates the request envelope, normalizes dates and units, checks essential fields, and asks for clarification only when a missing value prevents safe planning.
3. The Profile Agent converts the natural-language request into a structured user profile and separates non-negotiable constraints from weighted preferences.
4. The Supervisor creates the shared trip state, decomposes the request, records task dependencies, and dispatches retrieval, routing, budget, and context subtasks.
5. Retrieval agents collect candidate POIs, restaurants, accommodation, and travel information from configured databases, vector retrieval, and external tools, while storing source evidence and retrieval timestamps.
6. Candidates are normalized and filtered for destination, semantic relevance, category, opening hours, estimated duration, cost range, accessibility, and other user constraints.
7. The Route Agent requests travel-time and distance estimates for permitted modes, creates a pairwise route matrix, and produces an efficient daily visit sequence using shortest-path data and an itinerary-order heuristic.
8. The Budget and Constraint Agent calculates day and trip cost estimates, checks daily time, opening windows, travel duration, accessibility, and budget, and emits explicit violation objects rather than informal warnings.
9. The Itinerary Synthesis Agent assigns feasible candidates to days and constructs ordered time blocks for travel, activities, meals, check-in/check-out, and rest, together with estimates and explanations.

10. The Critic Agent checks unsupported facts, time overlap, excessive travel, duplicate POIs, closed venues, missing transfers, cost inconsistencies, weak evidence, and every declared hard constraint.
11. If a conflict is found, the critic returns a structured repair request to the Supervisor, which invokes the responsible agent to replace, resequence, shorten, or remove the affected item and then repeats validation.
12. After all hard checks pass, the system presents a validated day-wise itinerary with assumptions, source-aware recommendations, travel and cost estimates, and warnings for context that could not be verified.
13. User feedback, such as replacing an attraction or reducing walking, updates the profile and triggers partial replanning when possible; date, destination, or major budget changes may require complete replanning.

Structured profile: A typed profile will contain destination/city, origin, date interval, day count, budget and currency, traveler count/types, interests, food/accommodation preferences, permitted modes, mobility needs, daily start/end limits, and priority labels.

Common itinerary state: The shared state will hold candidate IDs, source/evidence metadata, coordinates, categories, hours, visit durations, estimated costs, accessibility attributes, a travel-time matrix, day assignments, selected modes, conflicts, and validation status.

Constraint policy: Hard conditions such as date range, closed hours, non-exceedable budget, accessibility, and maximum daily duration will be enforced before soft preference ranking. A soft preference may be relaxed only with a recorded rationale.

Error and uncertainty handling: Missing context will be represented as unavailable; stale or conflicting evidence will be flagged; tool failures will be logged; and the LLM will not be permitted to invent operational values to complete the schedule.

Prototype evaluation: Representative queries will vary budget, duration, group type, walking limits, interests, opening windows, and conflicting requirements. Planned measures include valid-field extraction, constraint-violation count, schedule feasibility, evidence coverage, route burden, and successful replanning.

Baseline comparison: Where feasible, the prototype will be compared with a simpler single-agent prompt pipeline and a non-agent heuristic to determine whether specialization and independent validation reduce violations; outcomes remain to be measured.

Chapter 5: Algorithms/Techniques

5.1 Algorithms / Techniques Used

1. LLM-based intent and preference extraction. Input will be free-form travel text; a constrained prompt will return a schema-validated profile. The Profile Agent will identify explicit values, inferred soft preferences, missing required fields, and confidence/evidence rather than relying on unstructured conversation.

2. Structured output generation. Pydantic/JSON schemas will define agent inputs and outputs for POIs, routes, budgets, daily time blocks, conflicts, and validation results. Schema validation will reject malformed output and allow the Supervisor to retry or route the task to a deterministic handler.

3. Agent task decomposition and orchestration. A state graph will encode dependencies and permitted transitions. Retrieval can run concurrently where appropriate, but route ordering will wait for candidate locations, and synthesis will wait for route and constraint results. The Supervisor will own retries, timeouts, and replanning.

4. Retrieval-Augmented Generation. The retrieval query will combine destination, dates, traveler profile, desired categories, and constraints. Retrieved records and tool responses will be attached to the trip state so that the LLM can synthesize recommendations from evidence and the critic can detect unsupported claims.

5. Semantic POI matching. Candidate descriptions and user interests may be embedded into vectors. Cosine similarity will estimate semantic relevance even when vocabulary differs, while metadata filters will enforce city, category, accessibility, price, and operating-date conditions before ranking.

6. Multi-criteria POI scoring. For user u and candidate p, a conceptual score is $S(p,u) = w_r R_p + w_q Q_p - w_c C_p - w_t T_p - w_d D_p - \lambda V_p$. All feature terms will be normalized to [0,1]; weights will be configurable and will not be described as experimentally optimized until evaluated.

7. Scoring-variable definitions. R_p is preference relevance; Q_p is a quality/popularity signal from the configured source; C_p is normalized cost burden; T_p is normalized visit-time burden; D_p is normalized detour/travel burden; V_p is the aggregate violation penalty; w_r, w_q, w_c, w_t, and w_d

are nonnegative trade-off weights; and lambda is the hard-constraint penalty multiplier.

8. Constraint-satisfaction checks. A candidate/time block will be feasible only if its visit interval lies inside its opening window, travel plus visit time fits the day, cumulative estimated cost remains within policy, accessibility conditions are satisfied, and mandatory breaks or daily limits are retained.

9. Map routing and shortest paths. The Route Agent will obtain mode-specific travel time and distance from a routing service when configured. A graph-based fallback can use Dijkstra or A* on available map data. Output will be a route matrix and source-aware transfer estimates, not invented geometry.

10. Visit-order optimization. Within each day, the sequencing problem resembles a Traveling Salesperson or Orienteering Problem with time windows. A practical prototype may use OR-Tools guided local search or an insertion/2-opt heuristic to reduce travel while retaining high-value POIs and mandatory time windows.

11. Multi-objective itinerary evaluation. A complete itinerary I may be compared through a vector containing preference satisfaction, total cost, total travel time, detour distance, evidence coverage, and violation count. Weighted scoring will support user priorities, while Pareto-style comparison can retain alternatives when one plan is cheaper but another better matches preferences.

12. ACO as a literature-informed alternative. The ACO approach in [4] uses pheromones, historical trip order, local search, and Pareto solutions to balance satisfaction, cost, and similarity. These ideas may inform visit-order search or alternative generation, but ACO will be implemented only if prototype complexity and data justify it.

13. Cooperative pathfinding concepts. Hybrid policy learning separates goal reaching from conflict resolution [3]. The proposed system will borrow the design principle of specialized resolution modules for route conflicts; it will not claim the paper's autonomous-vehicle MAPF policy as a tourist-itinerary algorithm.

14. Asynchronous MARL concepts. Region-based asynchronous coordination and critic-based validation in [5] suggest future congestion-aware routing at scale. Reinforcement learning is not required for the initial prototype, and no trained MARL model or traffic-simulation result is claimed.

15. Critic-based validation. The critic will recompute totals and evaluate a checklist independent of synthesis. Each finding will include severity, evidence, affected itinerary item, violated rule, and a recommended repair so the Supervisor can invoke a targeted revision.

16. Iterative replanning. The orchestrator will version the trip state, apply the smallest safe repair, rerun dependent checks, and stop only when hard constraints pass or the system can clearly explain why the request is infeasible under the supplied constraints.

17. Feedback and memory. Explicit user decisions, such as rejected categories, accepted trade-offs, walking tolerance, and preferred pace, may be stored with consent and reused as profile features. Historical feedback will be treated as preference evidence, not as a license to override current constraints.

18. Reliability and observability. Agent traces will record tool calls, latency, sources, schema failures, retries, conflict counts, and final validation. This supports debugging and planned comparison with single-agent and non-agent baselines without fabricating accuracy or efficiency claims.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

This synopsis proposes the design and implementation of an LLM-orchestrated multi-agent system for constraint-aware personalized trip itinerary planning. The project addresses the fragmentation of destination search, routing, cost control, scheduling, context, and verification by assigning them to specialized logical agents coordinated through a Supervisor and a shared structured trip state. The planned workflow will transform natural-language requirements into evidence-grounded candidates, optimize a day-wise sequence, check budget and operational feasibility, and return conflicts for targeted replanning.

The five objectives cover structured profile capture, modular agent design, contextual retrieval and ranking, multi-objective itinerary generation, and independent validation. The expected contribution is a modular and explainable prototype that can generate more coherent, personalized, and constraint-aware itineraries by combining collaborative agents, external tools, deterministic checks, and a critic loop. These are expected outcomes for Stage-I; completed deployment, commercial integration, user-study performance, and experimentally proven superiority are not claimed.

6.2 Future Scope

After the core prototype and baseline evaluation, the architecture can be extended in the following technically realistic directions:

- Real-time traffic and weather adaptation with event-triggered replanning.
- Live hotel, transport, attraction, and ticket availability with source timestamps.
- Seamless multi-modal optimization across walking, public transport, ride services, cycling, and private vehicles.
- Multi-city and multi-country planning with transfer days and border constraints.
- Multi-day hotel selection and optimization of hotel-to-POI travel burden.
- Group preference negotiation, fairness, and explicit compromise explanations.

- Accessibility-aware planning using verified mobility, terrain, entrance, and service data.
- Carbon-emission estimation and sustainability-aware route and mode comparison.
- Preference learning from historical feedback with consent, correction, and forgetting controls.
- Offline or privacy-preserving LLM and retrieval options for sensitive profiles.
- Multilingual and voice interaction with locale-aware dates, currency, and place names.
- Booking integration with human confirmation, payment isolation, and rollback handling.
- Dynamic disruption handling for closures, cancellations, delays, and unsafe weather.
- Formal evaluation against single-agent, rule-based, optimization-only, and non-agent baselines.

References

[1] Y. Liu, H. Chung, Z. Xu, K. Jang, S. Chen, and T. Chen, "LLM-TripPlanner: A Large-Language-Model-Based Agent for Personalized Trip Planning," in 2025 IEEE 28th International Conference on Intelligent Transportation Systems (ITSC), Gold Coast, Australia, 2025, pp. 927-934, doi: 10.1109/ITSC60802.2025.11423528.

[2] M. Dotoli, H. Zgaya, C. Russo, and S. Hammadi, "A Multi-Agent Advanced Traveler Information System for Optimal Trip Planning in a Co-Modal Framework," IEEE Transactions on Intelligent Transportation Systems, vol. 18, no. 9, pp. 2397-2412, Sep. 2017, doi: 10.1109/TITS.2016.2645278.

[3] A. Skrynnik, A. Yakovleva, V. Davydov, K. Yakovlev, and A. I. Panov, "Hybrid Policy Learning for Multi-Agent Pathfinding," IEEE Access, vol. 9, pp. 126034-126047, 2021, doi: 10.1109/ACCESS.2021.3111321.

[4] E. Saeki, S. Bao, T. Takayama, and N. Togawa, "Multi-Objective Trip Planning Based on Ant Colony Optimization Utilizing Trip Records," IEEE

Access, vol. 10, pp. 127825-127844, 2022, doi: 10.1109/ACCESS.2022.3227431.

[5] J. Yin, W. Rao, Y. Xiao, and K. Tang, "Cooperative Path Planning With Asynchronous Multiagent Reinforcement Learning," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 5016-5030, Jun. 2025, doi: 10.1109/TMC.2025.3526979.